

Muhammad Talha

Software Engineering

BSCS - 23122 (B)

Assignment 6

Part A: Theoretical Understanding

1) What is Selenium and why is it used in automated testing?

Selenium is a widely used open source automation framework specifically designed for testing web applications. It enables developers and testers to replicate real user interactions within a browser such as clicking on elements, typing into fields and navigation through pages. This capability makes Selenium an essential tool for validating the functionality and behaviour of web interfaces automatically.

Reason behind using Selenium in Automated testing:

The primary reason for using Selenium in automated testing is its support for cross browser and cross platform compatibility. It works seamlessly on major browsers like Google Chrome, Mozilla, Firefox, Safari and Microsoft Edge. It runs on operating systems such as Windows, Linux and macOS. This ensures that web applications function consistently across different environments.

Multi-language and Framework Support:

Selenium supports various programming languages including Java, Python, C# and Ruby, allowing integration with a wider range of technology stacks. It also works well with popular testing frameworks like JUnit, TestNG and Cucumber. These frameworks help in improving test management and enhancing reporting capabilities.

Benefits over Manual Testing:

Manual testing can be time-consuming, repetitive and error prone especially in large scale projects. Selenium overcomes the issue by allowing reusability of test scripts, parallel execution and integration with CI/CD pipelines, helping teams deliver high quality software faster.

Conclusion:

By automating functional and regression tests, Selenium ensures greater efficiency, accuracy and scalability in testing web-based applications. Its flexibility and broad compatibility make it a preferred choice for developers and testers aiming to improve user experience and application reliability.

2) Explain difference b/w absolute XPath, relative XPath and CSS selectors. Provide examples.

In UI Automation testing using Selenium, element locators are critical. XPath and CSS selectors are commonly used for locating elements on a webpage.

Absolute XPath:

Absolute XPath provides the complete path from the root `<html>` element to the desired element. It starts with a single slash `/` and is sensitive to any changes in page structure.

↳ Example: `/html/body/div[2]/form/input[1]`.

Although straightforward, it is brittle and not suitable for dynamic pages.

Relative XPath:

Relative XPath starts from any node in the document using `//`. It is more flexible and commonly used. It does not depend on the full hierarchy, making it more stable.

↳ Example: `//input[@id='username']`

Relative XPath can use attributes, text content and position to locate elements. Thus, making it more powerful in complex DOMs.

CSS selectors:

CSS selectors match elements using tags, names, classes, IDs and attributes. They are cleaner and faster in many cases especially for stable and simple web pages.

↳ Example: `input #username`, `button.submit-btn`.

In summary, CSS selectors are faster and cleaner than XPath.

Conclusion:

In short, absolute XPath is fragile while relative XPath and CSS selectors are preferred. CSS is better for speed and readability. XPath offers greater control in dynamic, complex structures.

3) Compare XPath and CSS selectors in terms of:

- * Readability
- * Performance
- * Flexibility

XPath and CSS selectors are two of the most widely used element locator strategies in selenium for automated web testing. They serve the same purpose but differ in syntax and capabilities.

* Readability:

CSS selectors are generally more readable and concise. Their syntax closely resembles style sheets; using in web development; making them easier for developers and testers to understand.

↳ Example: `input#username` is clearer than `input[@id='username']`.

XPath especially in complex queries can become lengthy and hard to interpret due to its hierarchical and expression based nature.

* Performance:

CSS selectors tend to be faster because modern browsers are optimized for CSS rendering. XPath is not natively supported in all browsers especially older versions of IE, which may result in slower element lookup in large and complex DOMs.

* Flexibility:

XPath is more powerful and flexible. It allows bidirectional navigation (up and down the DOM) using axes like `parent::`, `ancestor::`, etc. It can match text

Content, position or partial attribute values.

CSS selectors are limited to forward traversal and cannot search text content. It makes them less powerful in complex scenarios.

* Conclusion:

Use CSS selectors for simple and fast element matching in stable structures.

Use XPath for complex DOMs where detailed navigation and conditions are needed.

